Auditing and Repairing the Missing-Modality Generation Module of Multimodal Prompt Learning: Length-Bound Parameters, a Dead Prompt, and Real CLIP Features

Sujal Som (B24DS032), Ashish Kumar Dash (B24CS009), Tanmay Jaiswal (B24DS033)
CS550 Machine Learning, IIT Bhilai

***Abstract*—Multimodal Prompt Learning with Missing Modalities (MPLMM, ACL 2024) freezes a pretrained Multimodal Transformer and learns three small prompt types plus a "lightweight" generation module that synthesises absent modalities. Its authors state that the module may fail on raw features, which they expect to be less correlated across modalities. We audited the released code and tested that limitation on CMU-MOSI. *Audit.* One of the three prompt types is zero-initialised on both sides of a bilinear product, so it never receives gradient. It is all-zero in 55 of 55 trained runs, and the paper's Table 3 ablation cannot be reproduced from the released code; re-initialising its four tensors to $\mathcal{N}(0, 0.02)$ makes all four train. The released configuration reports the validation split instead of the test split, applies modality dropout to the evaluation splits, selects checkpoints on loss and stops early: it gives 57.55% against the reported 72.14%, and a corrected schedule recovers 67.51%. The code does not install from its stated requirements and needs two patches to load its own checkpoints on current PyTorch. The module's time-mixing layer is dense over *time*, so its size grows with sequence length, reaching 1,115,666 trainable parameters at native length, 104% of the frozen backbone. The upstream training loop and our fork's loop differ by 7.6 points on the same $\ell = 50$ cell, so we never compare numbers across loops. *Repair.* A length-invariant cross-attention operator matches the dense module at every length we tested, and is equivalent within the pre-registered $\pm$2-point margin at native length with 5.86$\times$ fewer parameters (pooled over two machines, TOST $p = 0.033$). Deleting the learned time-mixing layer altogether costs no detectable accuracy at $\ell = 200$. The logged generation-fidelity metric mostly measures padding layout. *Real foundation features.* We extracted CLIP ViT-B/32 features from all 2,199 raw MOSI clips. Swapping Facet for real CLIP vision causes no detectable accuracy change, and real CLIP vision is not less correlated with the text than Facet once speaker identity is removed. A label-aware projection prescribed by our own proposal turns out to be an invalid control: it memorises training-speaker sentiment.**

***Index Terms*—Multimodal sentiment analysis, missing modalities, prompt learning, reproducibility, parameter efficiency, CLIP, centered kernel alignment.**

I. Introduction

MULTIMODAL sentiment analysis predicts the sentiment of a short video clip from three co-occurring streams: the transcript, the voice and the face. In deployment, streams go missing: a microphone clips, a face leaves the frame, a transcript fails. Models trained on complete triplets then degrade. MPLMM [1] avoids elaborate reconstruction networks. It freezes a pretrained Multimodal Transformer (MulT) [2] and learns only three small prompt types: a generative prompt, a missing-signal prompt and a missing-type prompt. A deliberately simple generation module fills in absent modalities. The paper reports 72.14% average binary accuracy over six missing-modality cases on CMU-MOSI [3], against 70.84% for MMIN [4].

The authors name one limitation themselves: because the generation module is simple, "the performance of the model could degrade if we use raw features which are much more complicated and have weaker correlation between modalities than extracted features", and they leave this to future work [1]. Our project takes that limitation as its target. Reading the released implementation, we found a property the paper does not report: the module's main operator is a dense linear map over *time*, so its parameter count is bound to the sequence length. This predicts a second failure mode, driven by length rather than by inter-modal correlation.

We asked three questions, following our proposal (SOP):

- **Q1 (structural).** How do the module's parameters and inductive bias scale with sequence length?
- **Q2 (diagnostic).** Does performance degrade with sequence length, feature dimensionality or inter-modal correlation?
- **Q3 (repair).** Can a length-invariant operator replace the dense map without losing accuracy?

Contributions.

1) **Code audit.** We locate each fault in the released code at a line number (Table II): a prompt type that never trains, with a verified one-line repair; four training and evaluation faults that together cost 14.59 points; an undisclosed length scaling that makes the module larger than the backbone at native length; a configuration flag whose two branches are identical; and five environment blockers (Section V-A).
2) **Two training loops.** The upstream loop is bit-reproducible per machine and our fork's loop is not, and the two differ by 7.6 points on the same cell. We quantify this and keep every comparison inside one loop (Section V-B).
3) **Repair.** A length-invariant operator matches the dense module across a four-rung length ladder, and is statistically equivalent at native length at 5.86$\times$ fewer

parameters, replicated on two machines (Section V).
4) **Ablation.** Removing the learned time-mixing parameters entirely costs no detectable accuracy on the paper's features. The logged fidelity metric fails a negative control (Section VI).
5) **Real foundation features.** We built and verified a CLIP extractor on the raw MOSI video. The degradation the authors predict does not appear for vision, and the premise behind it, weaker inter-modal correlation, does not hold for vision (Section V-F).
6) **Negative results, reported.** Our own robustness prediction was wrong. A synthetic stand-in for foundation features was mislabelled. The label-aware control our proposal prescribed is invalid on speaker-independent splits.

A. Project details and team contributions

Criterion followed: the code is available, so we address a stated limitation and propose a solution. Target paper: Guo *et al.*, ACL 2024 [1]; code: https://github.com/zrguo/MPLMM.

- **Tanmay Jaiswal** (B24DS033, tanmay@iitbhilai.ac.in), feature pipeline and data:
  - MOSEI backbone pre-training;
  - the pooling-based sequence-length ladder (resampler with native-length identity check);
  - the 200-epoch launcher.

  The SOP also assigned him the CLIP extractor and merge utility; see the next entry for who built them.
- **Sujal Som** (B24DS032, sujals@iitbhilai.ac.in), model and training:
  - baseline reproduction runs;
  - the length-invariant operator (`lenmod/operator.py`);
  - the forward hooks for fidelity scoring (`lenmod/hooks.py`);
  - the 200-epoch training runs and operator ablations (arms C/D);
  - the CLIP extractor and merge utility code.
- **Ashish Kumar Dash** (B24CS009, ashishkd@iitbhilai.ac.in), evaluation and analysis:
  - the code audit: parameter audit, inductive-bias probes, the dead-prompt proof, and diagnosis of the evaluation-split fault;
  - the claims ledger and statistical protocol;
  - the fidelity audit (`tools/fidelity_audit.py`) and inter-modal CKA (`tools/intermodal_cka.py`);
  - the real-data CLIP run: extractor hardening, verification, speaker controls and the PCA control;
  - the figures and this report.

II. Related Work

Missing-modality learning. Imagination and reconstruction approaches learn to regenerate absent streams. MMIN [4], for example, uses cascaded residual autoencoders. These need sizeable reconstruction networks. Prompt-based alternatives began with missing-aware prompts for vision–language recognition [5], which build on prefix tuning [6] and visual prompt tuning [7]. MPLMM [1] adds a generation module, a real-versus-generated signalling prompt, and a compositional missing-type prompt that scales linearly in the number of modalities. A recent preprint questions whether generating missing modalities is needed at all, and evaluates before generating [8]. Our ablation in Section VI is a direct, controlled test of that question for MPLMM.

Features for multimodal sentiment. The CMU-MOSI and CMU-MOSEI [9] benchmarks are usually consumed as pre-extracted features: GloVe word vectors [10], COVAREP acoustic descriptors [11], and Facet facial action units [12]. Both non-text streams are hand-designed to track affect, which is the basis for the authors' expectation that raw or foundation-model features (wav2vec 2.0 [13], CLIP [14]) would be harder. End-to-end models on raw inputs exist [15]. Toolkits such as MMSA and MMSA-FET [16] distribute features and raw clips, but MMSA-FET has no CLIP extractor.

Representation similarity and equivalence testing. We measure generation fidelity and inter-modal coupling with linear centered kernel alignment (CKA) [17]. We state equivalence only with two one-sided tests (TOST) [18] against a pre-registered margin.

III. Proposed Methodology

A. The generation module as released

Fig. 1 shows the language-missing case. Two pointwise convolutions (`a2l`, `v2l`) mix channels at each time step. Their outputs are concatenated along *time* with a generative prompt of length $\ell_p = 16$. A transpose then moves the time axis into the channel slot of a `Conv1d(kernel_size=1)` layer (`l_avp`). That layer is therefore a dense, position-indexed linear map over time: every output position draws on every input position, with no weight sharing, locality or translation-equivariance. A learned missing-signal prompt is added (an addition, not a residual connection), and the result enters the frozen MulT backbone. A missing-type prompt is prepended for every sample. For a time-mixer with output length ℓ_{out} and total input length $\ell_{in} = \ell_p + \sum \ell_{src}$, the parameter count is

$$P_{\text{dense}} = \ell_{out}\,\ell_{in} + \ell_{out}. \tag{1}$$

Nine such layers exist: three two-source generators and six single-source generators. Each is bilinear in output and input length, hence quadratic when all modalities lengthen together.

B. Audit method

Every structural claim is checked by two independent routes before it is reported.

- **Parameter counts.** `tools/count_params.py` computes each count twice, by instantiating the real `PromptModel` and by the closed form of Eq. (1), and asserts that the two agree.
- **Inductive bias.** We perturb single time positions and single feature channels, and measure the diagonal mass of trained weights against chance.

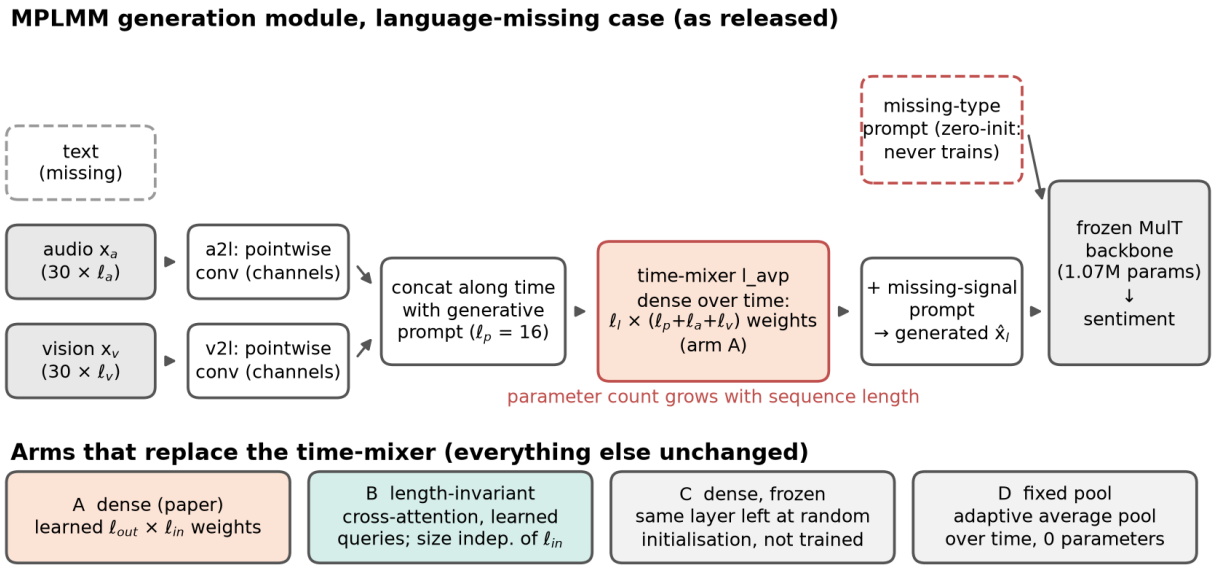


Fig. 1. The MPLMM generation module (language-missing case, as released) and the four arms that replace its time-mixing layer. Arms differ only in the time-mixer; prompts, channel mixers and the frozen MulT backbone are identical. The missing-type prompt is zero-initialised in a bilinear product and never trains (Section V-A).

- **Trainability of the missing-type prompt.** Five routes: hand derivation, a standalone reimplementation, a state-dict check, functional perturbation, and Adam at learning rate 1.
- **Reproduction.** The released configuration is compared against a corrected schedule, with each fault located at a line number.

C. The length-invariant operator (arm B)

We replace each of the nine time-mixers with cross-attention from a fixed set of learned queries to all available input tokens:

$$\hat{x} = \mathrm{GELU}\big(\mathrm{MHA}(Q, X, X)\big), \qquad Q \in \mathbb{R}^{\ell_q \times d_p}, \tag{2}$$

with $\ell_q = 50$ learned queries of width $d_p = 30$ and two attention heads. If $\ell_{out} \neq \ell_q$, the queries are linearly interpolated along time. The operator's own parameters do not depend on ℓ_{in}. The missing-signal prompts and missing-type coupling stay length-dependent, so total trainable parameters still grow, linearly. Only the operator is replaced; everything else in the model is unchanged.

D. Operator ablations (arms C and D)

Two arms separate "a dense map is needed" from "these parameters are needed".

- **Arm C** keeps the dense time-mixers frozen at their random initialisation, as a negative control. A fingerprint check asserts the weights are unchanged after backbone transfer.
- **Arm D** replaces them with a zero-parameter adaptive average pool over time, followed by the same GELU.

Both train exactly the same parameter set (86,491 at $\ell = 200$).

E. CLIP visual features from raw video (SOP item 2)

The extractor loads OpenAI's CLIP ViT-B/32 weights [14], verified by SHA-256 and frozen. For each segment it:

1) decodes the clip at 30 fps and native resolution with ffmpeg;
2) applies CLIP's official preprocessing (bicubic shorter-side resize to 224, centre crop, CLIP normalisation);
3) embeds every frame and L2-normalises the embedding.

It raises an error on any time window that decodes to zero frames, and stores the segment IDs alongside the features. Native per-frame embeddings are saved once, and each rung is then derived from them.

To keep layout fixed, the CLIP sequence occupies exactly the word slots that L1's word-aligned Facet features occupy: each clip's frames are split uniformly into k windows, where k is its number of words. The merge utility (SOP item 5) asserts that segment IDs match the reference dataset in order, and refuses inputs produced in synthetic mode. Text (GloVe) and audio (COVAREP) are copied unchanged from the reference file.

F. Fidelity and coupling metrics

The fork's hooks log linear CKA between generated and real features, flattened over time. Our audit tool rebuilds each checkpoint exactly as training did and must reproduce the stored values (tolerance 5×10^{-3}; achieved $\leq 1.8 \times 10^{-7}$) before computing anything new. It adds four corrected measures:

- CKA on a masked time-mean over real frames only;
- CKA of generated features against the 0/1 padding mask (a negative control);
- debiased CKA;

- *sentiment carried*: the Pearson r between the label and a ridge readout fitted on the *real* modality's training representations and applied to the generated one.

Inter-modal coupling is reported as masked, z-scored, clip-level linear CKA against a 200–500-permutation null. Where speaker identity could confound it, we centre each video's features on that video's own mean and permute within videos.

IV. Experimental Setup

A. Data

CMU-MOSI [3] has 2,199 opinion segments from 93 speakers, split speaker-independently into 1,284 / 229 / 686 segments (train / valid / test). We use the MulT feature release: GloVe-300 text [10], COVAREP-5 audio [11] and Facet-20 vision.

Length ladder. The unaligned audio/vision streams are average-pooled with fixed-interval windows to $\ell = 50/100/200$; the native rung is 375/500. This holds the extractor fixed and changes only length. Each rung file is checked against `data/ladder/SHA256SUMS` before use.

Aligned L1. The word-aligned 50-step file. It is 73–74% zero-padded in every modality.

Real CLIP rungs. Raw clips come from the MMSA release [16]: 2,199 MP4 segments in 93 folders, 518 MB. We checked them against the reference file: all 2,199 IDs matched, there were 0 split mismatches, the maximum label difference was 0.0, and clip durations were within 0.07 s of the reference end−start times (61 sampled clips). The official CMU server timed out from our machine. Three CLIP rungs were trained:

- L2-CLIP: 512-d;
- L2′-CLIP (PCA): PCA to 20-d fitted on train clip means without labels, rescaled to Facet's median row norm;
- L2′-CLIP (PLS): PLS to 20-d fitted on train labels, as the SOP prescribed; built once unscaled and once rescaled.

A separate synthetic "L2" rung (L1 features rotated into 768/512 dimensions plus orthogonal Gaussian noise) is reported as a noise stressor, not as foundation features (Section V-E).

B. Training protocol

All end-term runs share one configuration:

- one launcher (`run.py` via `launch_e200.py`), with the backbone (`pretrained/mosei.pt`) frozen;
- 200 epochs, learning rate 10^{-3}, batch 16 $\times$ 4 gradient accumulation (effective 64), learning-rate decay patience 30;
- modality dropout 0.7 during training.

Checkpoints are selected on the dropout-0.7 validation split for the length ladder and arms C/D at $\ell = 200$. The synthetic-L2 and real-CLIP grids select on complete-data validation (`--valid_drop_rate 0.0`). Each comparison uses one machine, one commit and one protocol. The real-CLIP grid ran entirely at commit `85eddde`. Our loop does not set `cudnn.deterministic`, so it is not bit-reproducible even on one machine; upstream `main.py` sets it and is (Section V-B).

C. Metrics and statistics

Primary accuracy. The *six-case forced mean Acc-2*: each of the six missing-modality cases is forced on all 686 test samples and the six accuracies are averaged. This is the paper-comparable number. Text is present in cases 1, 2 and 5 and absent in cases 0, 3 and 4. Case 6 is complete data.

Precision. One test sample is worth 0.15 points, and the binomial precision floor at $n = 686$ is 1.91 points.

Tests. We use 3 seeds per arm, the SOP's pre-registered fallback from 5. With 3 vs 3, the smallest attainable two-sided exact permutation p is 0.10, so large Welch t statistics from 2-degree-of-freedom variance estimates are not reported as small p-values. "No detectable difference" is never written as "equivalent". Equivalence is claimed only with TOST against the SOP's $\pm$2-point margin [18].

D. Resources

- **Hardware:** one NVIDIA RTX 3060 (12 GB). A 200-epoch run takes about 20–27 minutes at aligned length with two runs sharing the GPU, and about 2 h at native length. The native rung was replicated on a second machine.
- **Software:** PyTorch [19], scikit-learn and ffmpeg.
- **Grids:** run by detached, lock-guarded supervisors that restart after a reboot, so that runs do not depend on an open session.

In total there are about 98 converged 200-epoch runs, each evaluated on seven cases, plus 14 forty-epoch pilot runs.

V. Results and Analysis

A. Audit of the released code

A1 – The module outweighs the backbone it protects. Table I gives the trainable-parameter counts measured in trained runs. They match Eq. (1) exactly. At native length the dense module brings trainable parameters to 1,115,666, which is 104% of the frozen 1,071,211-parameter backbone. The paper describes prompts as 0.5–1% of the backbone and states a 5–10% envelope for total trainable parameters. The length scaling is not mentioned. Nine of the fifteen `MLPLayer` call sites are length-dependent; the discriminator is the third argument, `is_Fusion=True`. That flag selects two textually identical branches (`model.py:469--472`), so it has no effect: time-mixers and channel mixers are the same `Conv1d(kernel_size=1)`, and differ only in which axis the preceding transpose places in the channel slot. As a cross-check, a teammate's independent backbone count at MOSEI dimensions (1,073,731) differs from ours at MOSI dimensions (1,071,211) by exactly $2{,}520 = 30\times(74+35) - 30\times(5+20)$, the input-projection difference.

A2 – The "Conv block" encodes nothing local about time. Perturbing one input *time* position moves 50 of 50 outputs (375 of 375 unaligned). Perturbing one *feature* channel moves 1 of 30. On the converged checkpoint, the trained weights' diagonal-mass ratio to chance is 0.97 (range 0.91–1.02) across all nine layers. The layer is dense over time and pointwise over features, the opposite of what its shape

TABLE I
TRAINABLE PARAMETERS BY AUDIO/VISION LENGTH, MEASURED IN TRAINED RUNS (CLOSED FORM AND MODEL INSTANTIATION AGREE EXACTLY).

	$\ell = 50$	$\ell = 100$	$\ell = 200$	native
dense (paper)	88,141	155,241	349,441	**1,115,666**
% of backbone	8.2	14.5	32.6	**104**
invariant (ours)	97,471	109,471	133,471	190,471
% of backbone	9.1	10.2	12.5	17.8
dense / invariant	0.90	1.42	2.62	**5.86**

suggests. *Scope:* diagonal mass tests only near-diagonal concentration (band ± 3). Non-local structure, such as low rank or repeated rows, would not be detected, and no SVD or row-similarity check was run. The claim is "no local temporal structure", not "no temporal structure".

A3 – One of the paper's three contributions never runs. The batch missing-type prompt is the product of zero-initialised tensors (`model.py:225--230`). In a bilinear product each factor's gradient is proportional to the other factor, so zero stays exactly zero. Five independent routes agree:

- hand derivation;
- a standalone reimplementation, where both norms stay exactly 0 for 200 steps;
- a state-dict check: the four tensors are absent from the backbone checkpoint, so transfer cannot seed them;
- functional perturbation with $\mathcal{N}(0, 100)$ noise, giving maximum output change 0;
- Adam at learning rate 1 with weight decay.

The flag `missing_type_prompt_all_zeros` is true in **55 of 55** runs that record it, and in all 30 real-CLIP runs. The released code therefore cannot produce the paper's Table 3 ablation, because one ablated mechanism never activates. In every forward pass, 48 inert tokens (3×16) are prepended to the three backbone transformers. The magnitude, by the paper's own Table 3, is only about 0.2 points (71.94 vs 72.14), so this fault does not explain the reproduction gap.

Repair of the dead prompt. Re-initialising only the four tensors to $\mathcal{N}(0, 0.02)$ revives the mechanism (`tools/verify_dead_prompt_repair.py`; same seed, 50 Adam steps, all seven missing cases). In the released arm, every gradient, every displacement from initialisation, $\mathbf{m}_p$ and the prompt's effect on the output stay exactly 0. In the repaired arm, gradients become non-zero from step 2, once the missing-signal prompts leave zero. All four tensors then move (maximum displacement 0.022–0.029), $\max |\mathbf{m}_p|$ reaches 0.0101, and zeroing the prompt changes the output by up to 0.953. We verified the mechanism, not its accuracy. The paper's own ablation puts the effect at about 0.2 points, below our 1.91-point precision floor. A run directory named for the repair (`mt_repair_e200`) holds all-zero tensors, read directly from its checkpoint, so the patch was never applied there and no repaired accuracy is reported.

A4 – The released configuration does not reproduce its own number. Released defaults give 57.55%. A corrected schedule (batch 64, 200 epochs, decay patience 30) gives 67.51%, against the paper's 72.14%, closing the gap from 14.59 to 4.63 points. There are four compounding causes:

1) `train.py:160` reports the 229-sample validation split, not the 686-sample test split; line 140 computes the test loss and discards the predictions;
2) `get_loader` passes `drop_rate` to all three splits (`mosidata.py:55--56`), so evaluation and checkpoint selection run on a randomly corrupted signal;
3) checkpoints are selected, and the learning rate is decayed, on validation *loss* (`train.py:144, 154`), while the paper reports accuracy;
4) the defaults (`main.py:78--80`) give 40 epochs at decay patience 10, which stops before convergence and then decays the learning rate 100×, and `drop_rate` 0.6 where the paper's optimum is 0.7.

The corrected baseline is a single seed. We have not reproduced the paper under its stated 30-epoch setting, and why its schedule converges faster than ours remains open. The backbone is healthy: a from-scratch MulT reaches 80.18% on complete data, and freezing it to train 88,141 parameters costs only 2.1 points. The deficit concentrates where generation matters: -6.45 points versus the paper when text is absent, against -2.82 when text is present.

A5 – The code does not run as released. Five environment blockers recur on a clean install at upstream commit `c6f8d18`:

1) the README lists only Python $\geq$3.8 and PyTorch $\geq$1.8 and ships no requirements file, but `src/utils.py` imports `src/iemodata.py`, which needs `h5py`, `torchvision` and PIL even for MOSI;
2) checkpoints are pickled whole models, and on PyTorch $\geq$2.6 `torch.load` defaults to `weights_only=True`, so two sites (`utils.py:81`, `train.py:159`) must be patched before the code can load its own pretrained backbone or its own best checkpoint;
3) `ReduceLROnPlateau(verbose=True)` (`train.py:27`) raises `TypeError` on current PyTorch;
4) `nn.DataParallel` wraps the model whenever `batch_size`$>$10 (`train.py:66, 119`), whatever the GPU count;
5) the `drop_rate` default (0.6) differs from the paper's optimum (0.7).

Our fork applies items 2 and 3 as runtime shims in a wrapper and leaves `third_party/MPLMM` untouched. Upstream's README states that the repository is a reconstructed code base and invites error reports; we have prepared a correction note for the authors.

B. Two training loops are not interchangeable

The repository holds two training loops: upstream `main.py`→`src/train.py`, and our fork's `run.py`→`lenmod/trainer.py`, which every end-term grid uses.

- **Determinism.** Upstream sets `cudnn.deterministic=True` (`main.py:108`). Two 200-epoch upstream runs on one machine produced byte-identical checkpoints (MD5 `8840b40c…`). Our loop seeds every RNG but does not set the flag. Two same-seed, same-commit, same-machine runs already differ at epoch 1 (train loss 1.321796 vs 1.321862). Both

TABLE II
FAULTS LOCATED IN THE RELEASED CODE (UPSTREAM COMMIT `c6f8d18`), WITH EFFECT AND PROPOSED FIX.

#	location	fault	measured effect	proposed fix
1	`model.py:225--230`	missing-type prompt and its three couplings zero-initialised on both sides of a bilinear product	zero gradient; all-zero in 55/55 runs; Table 3 ablation not reproducible (≈0.2 pts by the paper)	initialise the four tensors $\mathcal{N}(0, 0.02)$ (verified to train)
2	`train.py:160` (and 140)	final metrics computed on the validation split	reported numbers are 229 validation samples, not the 686-sample test set	`evaluate(..., test=True)`
3	`utils.py get_data`; `mosidata.py:55--56`	modality dropout applied to valid and test	noisy selection and scheduler signal; evaluation not a fixed protocol	`drop_rate` only for train; evaluate each forced case
4	`train.py:144, 154`	selection and LR decay on validation loss; paper reports accuracy	part of the 14.59-point gap (with 2, 3, 5)	document, or select on complete-data validation accuracy
5	`main.py:78--80`	40 epochs, patience 10, `drop_rate` 0.6	released defaults 57.55% vs 72.14%; corrected schedule 67.51%	publish the exact configuration of the main results
6	`model.py:187--201`	dense time-mixers with $\ell_{out}\ell_{in}$ weights; size not disclosed	1,115,666 trainable at native length, 104% of the backbone	disclose; length-invariant operator (Sec. V)
7	`model.py:469--472`	`is_Fusion` branches identical	flag has no effect	remove, or implement the intended operator
8	README; `utils.py:81`; `train.py:27, 66, 119, 159`	missing dependencies; `torch.load` and scheduler incompatible with current PyTorch; unconditional `DataParallel`	does not install or load its own checkpoints without patches	requirements file with pinned versions; `weights_only=False`; drop `verbose`

earlier statements in our ledger, "bit-reproducible per machine" and "not bit-reproducible", are therefore true, of different loops.

- **Accuracy gap.** On the pooled $\ell = 50$ cell at 200 epochs, the upstream loop gives 68.93% (one run) and ours gives 61.30 ± 2.75 (three seeds), a 7.6-point gap. It concentrates in text-absent cases (60.77 vs 50.07) and complete data (77.13 vs 70.12). The upstream run's log does not record its seed or dropout rate (defaults 666 and 0.6), so the gap is attributable to the loop *and* its configuration, not to the loop alone.

Consequences: A4's 67.51% (upstream loop, aligned data) is *not* comparable with Table III or any other end-term number (our loop). Within-grid comparisons, including the equivalence result, are unaffected, because each grid uses one loop, one commit and one machine.

C. Length ladder: the repair holds at every length

Fig. 2 and Table III summarise the four rungs.

- **No detectable difference at any length.** Two-way ANOVA over $\ell \leq 200$ gives arm $p = 0.18$ and arm×length $p = 0.43$. The $\ell = 50$ gap of +2.74 exceeds 2 points but has $p = 0.22$, so it is not claimed.
- **Equivalence at native length.** We pooled the box and an independent second machine (different install and commit) with machine as a block. The invariant − dense difference is $+0.04$ points, 90% CI $[-1.59, +1.67]$, inside the ±2 margin: TOST $p = 0.033$. Each machine alone gives TOST $p = 0.069$ and 0.078, and the arm×machine interaction is $p = 0.88$ (Fig. 3). This is the project's only equivalence claim.
- **Parameter growth.** Invariant parameters grow exactly as $240\ell + 85{,}471$ (zero residual at all four rungs), against roughly ℓ^2 for dense. The curves cross at 60 frames, which is why the invariant arm is slightly *larger* at $\ell = 50$.

TABLE III
SIX-CASE FORCED MEAN ACC-2 (%), MEAN ± SD OVER 3 SEEDS; p IS THE EXACT TWO-SIDED PERMUTATION TEST (WELCH FOR $\ell \leq 200$).

rung	dense	invariant	B−A	p
$\ell = 50$	61.30 ± 2.75	64.04 ± 1.25	+2.74	0.22
$\ell = 100$	65.93 ± 1.85	66.54 ± 0.32	+0.61	0.63
$\ell = 200$	66.65 ± 2.07	66.85 ± 1.32	+0.20	0.89
native (box)	66.13 ± 1.43	66.27 ± 0.70	+0.14	0.80
native (2nd machine)	66.01 ± 1.58	65.94 ± 0.75	−0.08	0.94
pooled native, machine-blocked: +0.04, 90% CI [−1.59, +1.67], TOST $p = 0.033$				

Our robustness prediction was wrong. The SOP predicted that dense would degrade at long lengths. It does not. Both arms *improve* by about +2.0 points per doubling over $\ell = 50$–200 (OLS $p = 0.0018$, $n = 18$), and dense at native (66.13) is level with $\ell = 200$ (66.65). The efficiency argument survives; the robustness argument does not. Inter-modal coupling stays flat along the ladder, so this is a length result, although longer rungs also mean less pooling.

Noise floor. The native seed-1 configuration was run three times with no seed change: on the box at commit `ebbcb4e`, on the box at `f3cebe4`, and on the second machine. This gives dense 68.57 / 67.78 / 67.58 and invariant 66.41 / 66.87 / 66.67. The three runs span commits and machines, so they bound the combined spread at about 1 point rather than isolating GPU nondeterminism. Single-run gaps under about 1 point are not interpretable.

The $\ell = 50$ complete-data inversion. In 3 of 6 $\ell = 50$ runs, complete-data accuracy is more than 2 points below the text-present missing cases (dense seed 1: 66.01 vs 70.73; invariant seeds 1 and 3: 63.26 vs 75.05 and 63.57 vs 74.49). Every $\ell = 100$, 200 and native run is within 1.2 points. This is not a new anomaly. Our ledger (CLAIMS §4.10) attributes the same inversion, seen earlier on undertrained checkpoints, to undertraining. It is confined to the cell where our loop trails the upstream loop by 7.6 points, and the upstream $\ell = 50$ run

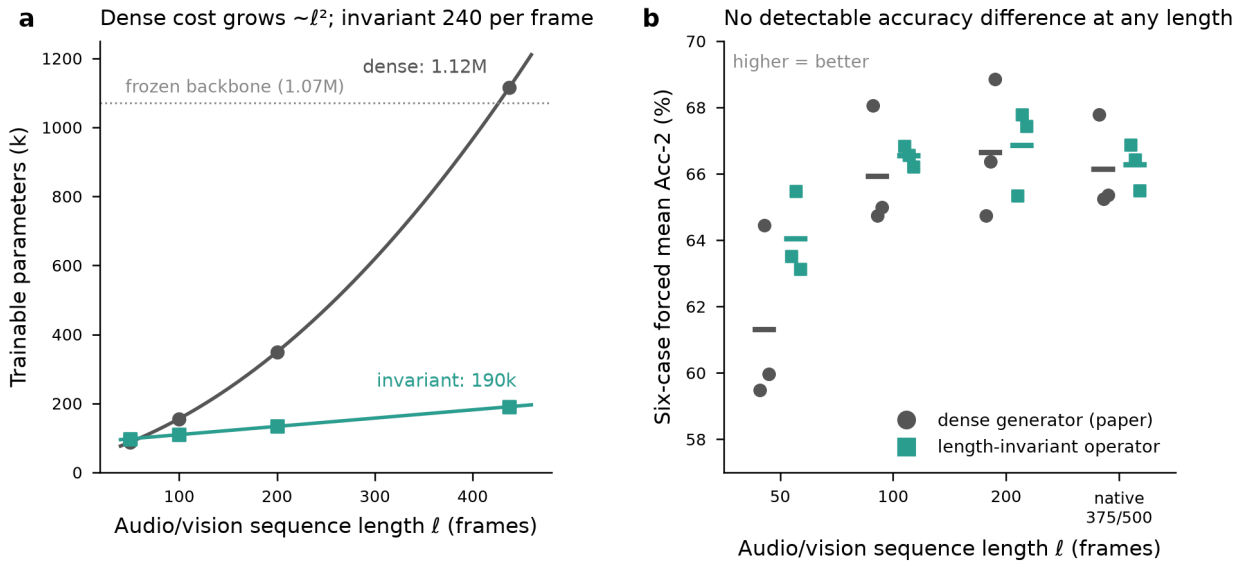


Fig. 2. (a) Trainable parameters versus audio/vision length: dense grows as $\sim \ell^2$ and exceeds the frozen backbone at native length, while the invariant operator grows by 240 parameters per frame (exact fit). (b) Six-case forced mean Acc-2 per seed, $n = 3$ per arm per rung, on one machine; native uses commit `f3cebe4`. Bars are arm means.

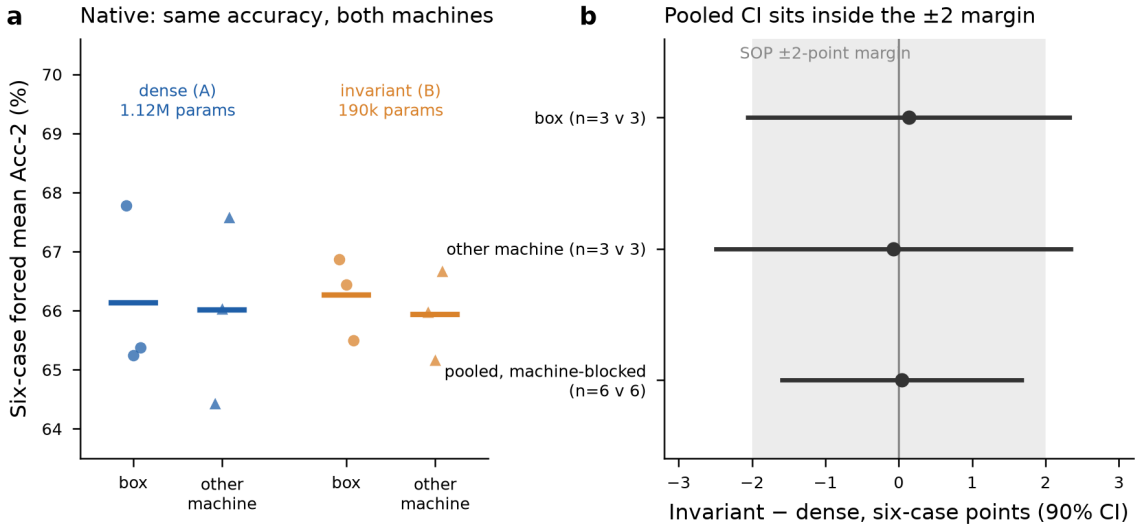


Fig. 3. Native length (375/500), $n = 3$ per arm per machine. (a) Per-seed accuracy on two independent machines. (b) Invariant − dense with 90% confidence intervals; only the machine-blocked pool sits inside the SOP's ±2-point equivalence margin.

shows no inversion (77.13 vs 77.08). Best epochs here (69–169) lie inside the 200-epoch budget, so if this is undertraining, the cause is the loop at this rung rather than the epoch cap. We have not tested that on these checkpoints.

D. Generation fidelity is mostly padding layout

Fig. 4 and Table IV show the fidelity audit. MOSI audio/vision is 86–90% left-padding, and the number of real frames (clip duration) is shared across modalities, with valid-frame counts correlated at $r = 0.992$–0.998. Two consequences follow:

- The SOP's literal "mean-pooled" coupling measure gives about 0.53 at every rung, about 14× the real-frame value.
- The logged time-flattened CKA rewards reproducing *where the padding is*. On real frames, the dense advantage for audio/vision is not distinguishable from zero.

For text, dense is geometrically closer to the real features, yet the invariant operator's generated text carries more sentiment. Neither carries much: at most 28% of the real text's sentiment correlation. No CKA variant predicts accuracy: Spearman $\rho = -0.005$, -0.138 and -0.014 for the logged, masked and debiased variants over 20 runs. The logged dense advantage does replicate across independent run sets: 1.84× in the 14 forty-epoch pilot runs (0.3010 vs 0.1632) and 2.16× in the 18 converged runs (0.318 vs 0.147). A replicated padding artefact, however, is still not evidence of fidelity.

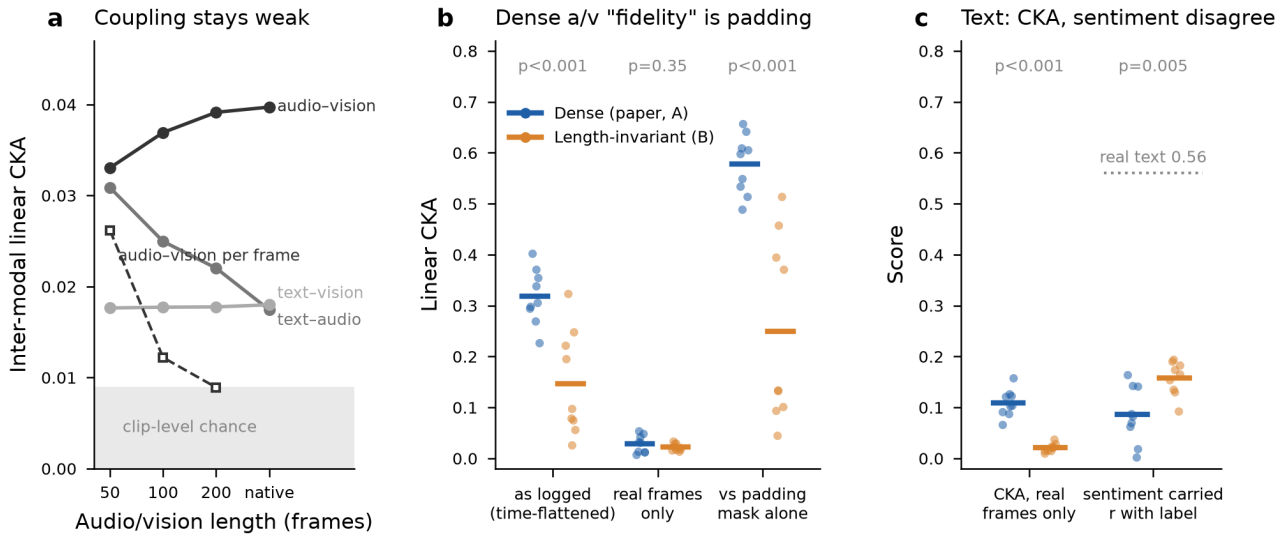


Fig. 4. (a) Inter-modal linear CKA along the pooled ladder (masked, z-scored, clip-level; grey band = 95th percentile of a permutation null): coupling is weak at every length. (b) Generated-vs-real audio/vision fidelity: the logged advantage of dense disappears on real frames, and dense instead reproduces the padding mask. (c) Generated text: geometric fidelity and sentiment content disagree. Nine runs per arm ($\ell = 50/100/200 \times 3$ seeds).

TABLE IV
FIDELITY OF GENERATED FEATURES, LENGTH LADDER $\ell \leq 200$, 9 RUNS PER ARM (WELCH p).

measure	dense	invariant	p
a/v CKA as logged (time-flattened)	0.318	0.147	<0.001
a/v CKA, real frames only	0.028	0.022	0.35
a/v CKA vs. 0/1 padding mask	0.578	0.249	<0.001
text CKA, real frames	0.109	0.021	<0.001
text sentiment carried (r)	0.086	0.158	0.005
reference: real text	0.562	0.560	0.82

E. A synthetic "L2" rung is a noise stressor, not foundation features

The L2/L2′ files we first used for SOP Q2 contain no foundation-model features. They take the L1 COVAREP/Facet features, rotate them into 768/512 dimensions with random orthonormal bases, and add temporally smoothed Gaussian noise (std 0.8) in the orthogonal complement. The builder labelled them "wav2vec2" and "CLIP"; we corrected the labels and treat the rung only as a controlled stressor.

Under that stressor the invariant operator degrades less than dense: 58.41 ± 0.62 vs 54.54 ± 0.33. That is +3.87, with complete rank separation, exact $p = 0.10$ (the $n = 3$ floor), and 2.0× the precision floor. The coupling "gate" originally computed for these files was itself a padding artefact. Its L1 values of 0.496 / 0.678 (audio–text / vision–text) fall to 0.023 / 0.031 on real frames, so the original gate reading is retracted.

F. Real CLIP features from raw MOSI video

Extraction and verification.

- All 2,199 clips were extracted in 1,079 s with 0 failures.
- On a real 640×360 frame, the extractor matches reference CLIP `encode_image` at cosine 0.99886.
- Zero-shot, 2,198 of 2,199 middle frames are classified as "a photo of a person talking to the camera".
- A "happy person" minus "sad person" prompt score correlates with the test sentiment label at $r = 0.166$ ($p = 1.2 \times 10^{-5}$).

CLIP features are mostly speaker identity (Fig. 5a). 92.7% of clip-level CLIP variance lies between speakers, against 62.8% for Facet, 40.5% for COVAREP and 11.1% for GloVe. A raw ridge probe on CLIP therefore transfers nothing to unseen speakers (test $r = -0.004$, 95% CI $[-0.074, 0.070]$), while Facet gives 0.178. After per-speaker centring, CLIP gives 0.072 and Facet 0.032. The confidence intervals overlap, so this comparison is descriptive.

Real CLIP vision is not less correlated with the text than Facet (Fig. 5b). Most raw vision–audio coupling is shared speaker identity: after per-speaker centring, the excess over a within-video null falls from 0.185 to 0.003 for CLIP ($p = 0.10$) and from 0.092 to −0.001 for Facet ($p = 0.65$). Vision–text coupling survives centring for both and is slightly higher for CLIP: 0.020 vs 0.014, both $p = 0.002$. This directly tests the premise of the paper's Limitations section on real foundation features, and for vision the premise is not supported.

No detectable accuracy change (Table V, Fig. 5c). Replacing Facet with real CLIP vision changes the dense arm by −0.93 points ($p = 0.6$) at 512 dimensions and −0.35 ($p = 0.9$) at 20 dimensions with the label-free PCA projection. Invariant − dense is null at every CLIP rung. The +3.87 synthetic-L2 advantage therefore does *not* replicate on real features. Best epochs come earlier on CLIP (about 27 vs 45–53 at L1), consistent with the speaker signal being fitted quickly.

VI. ABLATION STUDY

A. Is the learned time-mixing layer needed? (arms C, D)

At $\ell = 200$ (Fig. 6) the four arms score:

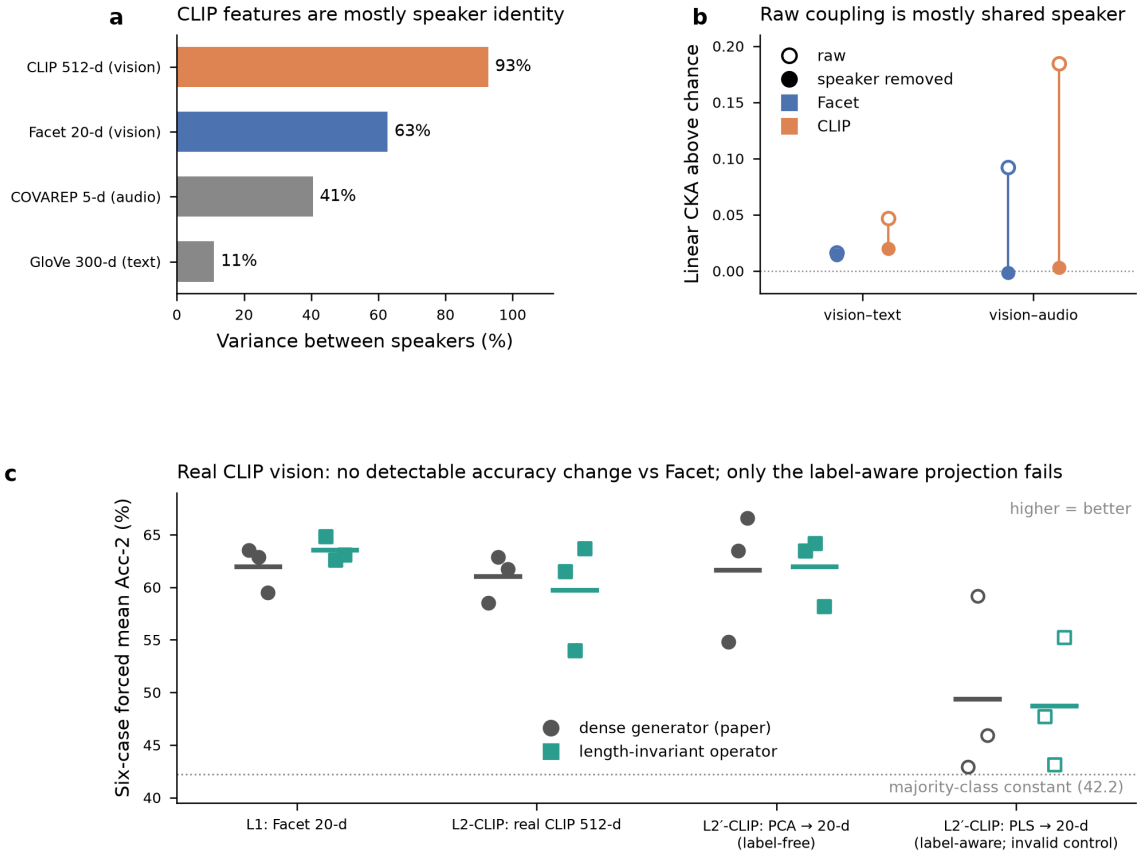


Fig. 5. Real CLIP ViT-B/32 features extracted from all 2,199 raw CMU-MOSI clips. (a) Share of clip-level variance between speakers (test split, 31 speakers). (b) Inter-modal linear CKA above a permutation null, raw (hollow) and with speaker removed (filled; null permuted within video). (c) Per-seed accuracy, $n = 3$ per arm; hollow markers mark the label-aware PLS rung, an invalid control. The first, unscaled PLS build is listed in Table V but not plotted.

TABLE V
REAL-CLIP RUNGS: SIX-CASE FORCED MEAN ACC-2 (%), MEAN ± SD OVER 3 SEEDS; ONE COMMIT (85EDDDE), COMPLETE-DATA VALIDATION. p IS THE EXACT PERMUTATION TEST.

rung	vision	dense	invariant	B−A (p)
L1	Facet 20-d	61.96 ± 2.15	63.51 ± 1.15	+1.55 (0.5)
L2-CLIP	CLIP 512-d	61.03 ± 2.25	59.73 ± 5.09	−1.30 (0.8)
L2′-CLIP	PCA-20	61.61 ± 6.10	61.95 ± 3.28	+0.34 (0.9)
invalid control (label-aware projection, Section VI)				
L2′-CLIP	PLS-20	42.68 ± 0.79	48.64 ± 8.87	–
L2′-CLIP	PLS-20, rescaled	49.34 ± 8.62	48.72 ± 6.11	−0.62 (1.0)

- A, dense (paper): 66.65 ± 2.07;
- B, invariant: 66.85 ± 1.32;
- C, dense frozen at random initialisation: 65.15 ± 0.52;
- D, zero-parameter fixed pool: 66.98 ± 0.79.

No arm differs from A: one-way ANOVA $F = 1.25$, $p = 0.354$. Deleting the 262,950 learned time-mixing parameters (D vs A) changes accuracy by +0.33 (Welch $p = 0.82$, 90% CI $[-2.90, +3.56]$). On the paper's extracted features, the learned time-mixing layer buys no detectable accuracy. The fidelity audit points the same way. Logged audio/vision CKA is 0.292 for trained dense, 0.289 for random-frozen dense and 0.298 for the fixed pool, and all three match the padding mask equally (0.54–0.59). The logged metric therefore cannot distinguish a trained generator from an untrained one. Descriptively, D exceeds C by +1.83 (Welch $t = 3.37$, $p = 0.035$), and C's deficit is confined to text-absent cases (52.41 vs 55.45 for A; text-present 77.90 vs 77.85), consistent with A4's larger deficit where text is absent. This is one of several pairwise comparisons under a null omnibus test and does not survive Holm correction. Arm D selects late (best epochs 197, 132 and 158), so 200 epochs may still truncate it. Arms C and D ran at project commit `73a2769` and arms A and B at `ebbcb4e`. The intervening edits are additive, but the comparison departs from our one-commit rule.

Boundary. Under the synthetic noise stressor, the same ablation does cost accuracy. Dense scores 54.54, against C 52.63 and D 52.49, and all six C/D runs fall below every dense run. That gap is only 1.0–1.1× the precision floor, so it is not established. The claim is therefore bounded: *on the paper's features*, the learned time-mixer is unnecessary.

B. Label-aware versus label-free projection

The SOP's L2′ control projects high-dimensional features to Facet's 20 dimensions with PLS fitted on the training labels. On MOSI this is invalid (Table VI). Because the splits are speaker-independent, PLS memorises each *training speaker's* average sentiment, and that does not transfer to unseen speakers. The model takes this shortcut and ignores

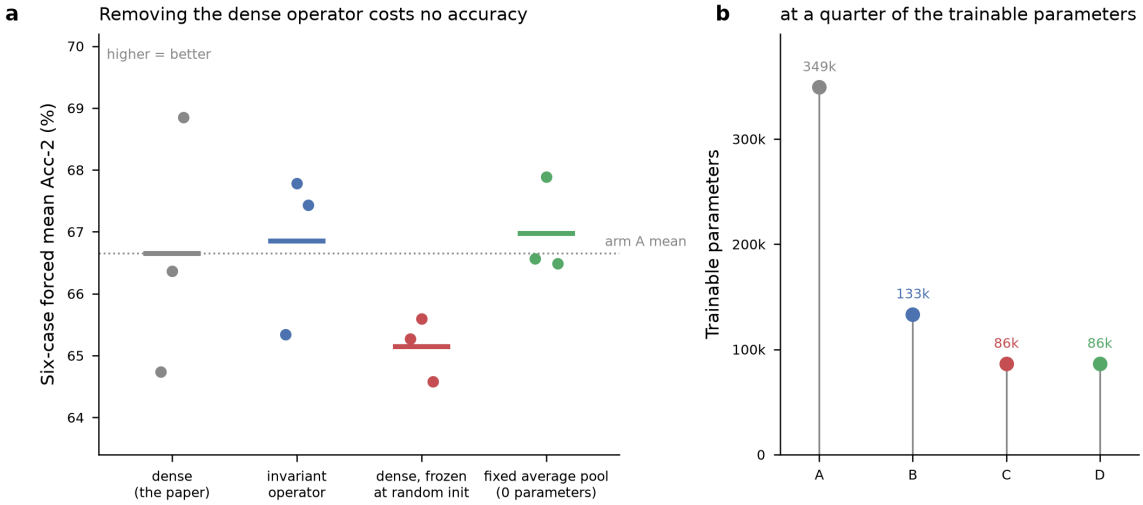


Fig. 6. Operator ablation at $\ell = 200$, $n = 3$ per arm, one machine. (a) Six-case forced mean Acc-2 per seed; dotted line = arm A mean. (b) Trainable parameters. Arms C and D train the identical 86,491-parameter set; C keeps 262,950 dense time-mixing weights frozen at random initialisation, and D removes them.

TABLE VI
LINEAR FIT OF CLIP-LEVEL VISION FEATURES TO THE LABEL (FITTED ON TRAIN).

features	train r	valid r	test r	speaker-level train r
CLIP $\rightarrow$ PLS-20	0.673	0.006	0.029	**0.991**
synthetic L2′ (PLS-20)	0.671	−0.054	0.068	**0.911**
CLIP $\rightarrow$ PCA-20	0.345	0.200	0.053	–
Facet 20-d (L1)	0.341	0.237	0.178	0.615

the text: text-present cases drop 18–25 points. The PLS scores were also about 13× Facet's scale. Unscaled, all three dense seeds sat at the 42.23% majority-class constant (best epochs 1–2); rescaling removed the constant but not the failure. A label-free PCA projection at the same dimension and scale trains normally (61.61 / 61.95). The synthetic L2′ rung shares the flaw, which plausibly explains its unstable seeds.

C. Checkpoint-selection protocol and training budget

Checkpoint selection. Selecting on complete-data rather than dropout-0.7 validation stabilised the synthetic-L2 arms. Dense went from 54.01 ± 1.07 to 54.54 ± 0.33, and invariant from 56.90 ± 0.81 to 58.41 ± 0.62. It did not rescue L2′: it changed which seed collapsed.

Training budget. Training for 200 rather than 40 epochs changed the six-case mean by only +0.67 points across 14 seed-matched cells ($p = 0.17$). This retracts an earlier claim that 40 epochs cost about 7 points. That claim set a 40-epoch run of our loop (61.89) against a 200-epoch run of the upstream loop (68.93), so most of its "7 points" is the loop gap of Section V-B. The pilot's asymmetric-truncation hypothesis also failed. Invariant runs had peaked later at 40 epochs (mean best epoch 35.0 vs 27.3), which predicted that more budget would favour the invariant arm. Instead, 200 epochs moved dense by +1.27 and invariant by −0.52 at $\ell = 100$, and by +0.75 and +0.60 over all 14 matched cells.

Preprocessing. Replacing the extractor's original 224×224 squash with CLIP's official shorter-side resize and centre crop raised agreement with reference CLIP on 4:3 frames from 0.951–0.986 to 0.999.

VII. DISCUSSION

Fault in the paper, or in the code? *In the code* — proven:

- one prompt type is structurally untrainable (A3);
- the evaluation reports the wrong split and the defaults undertrain (A4);
- the module's cost scales with length in a way the paper does not disclose (A1);
- the released code does not install from its stated requirements or load its own checkpoints on current PyTorch (A5).

Each fault, its location and a proposed fix are collected in Table II and in a separate correction note addressed to the authors. We did *not* show that the paper's 72.14% is wrong; the remaining 4.63-point gap is open. The defensible claim is: *the released implementation cannot reproduce the paper's ablations, and its generation module's learned, length-bound parameters can be replaced or removed at no detectable accuracy cost.*

Why does generation matter so little? Three lines of evidence agree:

1) the time-mixer can be deleted (arm D) at no detectable cost;
2) its "reconstruction" mostly reproduces padding layout;
3) what it generates carries at most a quarter of the real text's sentiment signal, and essentially none of the real audio/vision signal.

Text dominates MOSI: GloVe alone gives a probe r of 0.58, against 0.18–0.24 for Facet and COVAREP. So the benefit of the module is small wherever text is present.

SOP Table III reading. For vision on real CLIP features, accuracy and real-frame fidelity are both stable: masked a/v

TABLE VII
PROPOSED (SOP) VERSUS DELIVERED.

commitment	status	evidence
Q1 structural scaling	yes	Tab. I, A2
Dead-prompt repair	mechanism	A3; accuracy not measured
Reproduction gate (72.14 ± 2)	missed	67.51; causes located (A4)
Q2 length ladder	yes	Fig. 2; prediction refuted
Q2 features / correlation	vision only	real CLIP rungs (Sec. V-F)
Q3 length-invariant operator	yes	equivalence at native
Operator ablations	partly	arms C, D; no strided/dilated conv
Fidelity G (primary)	corrected	audit, Tab. IV
Margin M (primary)	no	no backbone-only control
CLIP extractor + merge	yes	2,199 clips, verified
5 seeds	3 seeds	pre-registered fallback

CKA for the dense arm is 0.026 at L1 and 0.026 at L2-CLIP, and inter-modal coupling is not lower. This matches the row "G, A both stable $\rightarrow$ concern unfounded". The margin M over a backbone-only control was not measured.

VIII. Limitations

- **Reproduction gap.** The 4.63-point gap to the paper is unexplained, and the corrected baseline rests on one seed.
- **Modalities.** Only vision was replaced by a foundation feature; audio stays COVAREP, so the real-CLIP rungs mix feature pipelines. The frozen backbone was pre-trained on Facet-style MOSEI features, so only the input projection adapts to CLIP.
- **Statistical power.** With $n = 3$ seeds, most results are "no detectable difference". Equivalence is shown only at native length and only on the two-machine pool; at $\ell = 200$, about 14 seeds per arm would be needed.
- **Scope.** CMU-MOSI only; no CMU-MOSEI fine-tuning, CH-SIMS or IEMOCAP.
- **Repair cost.** The invariant operator is 12.5% of the backbone at $\ell = 200$ and 17.8% at native, above the paper's 5–10% envelope, and slightly larger than dense below 60 frames. The claim is "5.86$\times$ cheaper than dense", not "within budget".
- $\ell = 50$ **inversion.** Linked to undertraining and to the loop gap (Section V), but not tested on these checkpoints.
- **Two loops.** The 7.6-point gap between the upstream loop and ours is measured on one cell, against one upstream run whose seed and dropout are unrecorded; its cause is not isolated. A4 and the end-term grids are not comparable.
- **Repair untrained.** The dead-prompt repair is verified mechanically, not trained to convergence; its expected effect (about 0.2 points) is below our precision floor.
- **Locality scope.** A2's diagonal-mass test cannot detect non-local temporal structure; the SVD/row-similarity check is not done.
- **Protocol differences.** The ladder (dropout-0.7 selection) and the CLIP grids (complete-data selection) use different checkpoint-selection protocols. Our loop is not bit-reproducible, and arms C/D ran at a different commit from arms A/B.
- **Layout approximation.** The CLIP word-slot layout uses a uniform split of each clip, not word timestamps; agreement with Facet's slots is 99.1–99.5%.

IX. Conclusion and Future Work

MPLMM's released generation module is neither as light nor as necessary as presented. Its time-mixing layer grows with sequence length past the size of the frozen backbone, one of its three prompt types never trains (a one-line re-initialisation revives it), its evaluation reports the wrong split under randomly corrupted inputs, and it does not run on current PyTorch without patches. Its training loop and our fork's loop are not interchangeable, so every comparison here stays inside one loop. Replacing the time-mixer with a length-invariant operator preserves accuracy at every length and is equivalent at native length with 5.86$\times$ fewer parameters. Deleting the learned time-mixer costs no detectable accuracy on the paper's features. On real CLIP features extracted from the raw MOSI video, the failure the authors predicted does not appear for vision, and the weaker cross-modal correlation it rests on is not observed.

Future work:

- extract wav2vec 2.0 audio to complete the foundation-feature rung;
- add backbone-only and generator-bypassed controls to measure the margin M;
- use speaker-grouped cross-fitting for any label-aware projection;
- raise seeds to establish equivalence at $\ell \leq 200$;
- test on MOSEI and CH-SIMS;
- train the repaired missing-type prompt to convergence;
- run the upstream and fork loops on one cell with matched seed and dropout to isolate the 7.6-point gap;
- run the SVD/row-similarity check that closes A2's scope;
- submit the accompanying correction note to `zrguo/MPLMM` as issues, with patches.

References

[1] Z. Guo, T. Jin, and Z. Zhao, "Multimodal prompt learning with missing modalities for sentiment analysis and emotion recognition," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, 2024, pp. 1726–1736, code: https://github.com/zrguo/MPLMM.

[2] Y.-H. H. Tsai, S. Bai, P. P. Liang, J. Z. Kolter, L.-P. Morency, and R. Salakhutdinov, "Multimodal transformer for unaligned multimodal language sequences," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 6558–6569.

[3] A. Zadeh, R. Zellers, E. Pincus, and L.-P. Morency, "MOSI: Multimodal corpus of sentiment intensity and subjectivity analysis in online opinion videos," *arXiv preprint arXiv:1606.06259*, 2016.

[4] J. Zhao, R. Li, and Q. Jin, "Missing modality imagination network for emotion recognition with uncertain missing modalities," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021.

[5] Y.-L. Lee, Y.-H. Tsai, W.-C. Chiu, and C.-Y. Lee, "Multimodal prompting with missing modalities for visual recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[6] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021, pp. 4582–4597.

[7] M. Jia, L. Tang, B.-C. Chen, C. Cardie, S. Belongie, B. Hariharan, and S.-N. Lim, "Visual prompt tuning," in *European Conference on Computer Vision (ECCV)*, 2022.
[8] "Evaluation before generation for multimodal learning with missing modalities," *arXiv preprint arXiv:2604.05558*, 2026.
[9] A. B. Zadeh, P. P. Liang, S. Poria, E. Cambria, and L.-P. Morency, "Multimodal language analysis in the wild: CMU-MOSEI dataset and interpretable dynamic fusion graph," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2018, pp. 2236–2246.
[10] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532–1543.
[11] G. Degottex, J. Kane, T. Drugman, T. Raitio, and S. Scherer, "COVAREP – a collaborative voice analysis repository for speech technologies," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 960–964.
[12] P. Ekman and W. V. Friesen, *Facial Action Coding System: A Technique for the Measurement of Facial Movement*. Palo Alto, CA: Consulting Psychologists Press, 1978.
[13] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020, pp. 12 449–12 460.
[14] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.
[15] W. Dai, S. Cahyawijaya, Z. Liu, and P. Fung, "Multimodal end-to-end sparse model for emotion recognition," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics*, 2021.
[16] H. Mao, Z. Yuan, H. Xu, W. Yu, Y. Liu, and K. Gao, "M-SENA: An integrated platform for multimodal sentiment analysis," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 2022, mMSA and MMSA-FET toolkits: https://github.com/thuiar/MMSA.
[17] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of neural network representations revisited," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 3519–3529.
[18] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, 1987.
[19] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

Appendix

AI tools, prompts and verification

The instructor permitted AI-assisted coding for this project.

Platforms.

- Claude (chat) for literature scoping and experimental design.
- Claude Code for tracing the MPLMM, MMSA, MMSA-FET and CMU-MultimodalSDK codebases.
- Claude Science (an agentic analysis environment with SSH access to the project GPU box) for the end-term evaluation:
  - the fidelity and CKA audits and the statistics;
  - the length-ladder, native, arm C/D and CLIP grids, driven by detached supervisors;
  - extractor hardening and verification;
  - figures and this report's numbers.

Teammates' own AI usage for their components (operator, training runs, ladder resampler, extractor code) is not recorded here; each member should add theirs.

Representative prompts (Ash, end-term):

- "What should be our end-term deliverable that will be solid considering our project status and goals, what we've proved till now, the numbers we have…"
- "Verify the CLIP extractor is working and is well."
- "Load real CMU-MOSI video and test this CLIP extractor on it; I want real results to showcase the CLIP extractor as a finding."
- "Does this evaluation prove a new finding or boost our case, or should we wait for the L2′ results?"
- "Give a comprehensive project report summarising all key findings and the data behind them, following the given format."

Verification protocol and AI errors caught. Every number in this report is read from a run record or results file, and every claim in the project's claims ledger carries a status tag. AI-generated errors that were caught and retracted:

- an inverted axis ("temporal receptive field of one position"), which reached the proposal abstract;
- "40 epochs cost ∼7 points" (a cross-entry-point comparison);
- a padding-inflated coupling gate (L1 CKA 0.496/0.678, real frames 0.023/0.031);
- a Welch $p = 0.002$ reported at $n = 3$, where the exact floor is 0.10;
- "dense is best at native";
- a synthetic rung mislabelled as wav2vec2/CLIP features;
- our own label-aware PLS control, which leaks training-speaker sentiment;
- "1.42 points of GPU nondeterminism" (the 67.51/68.93 difference is data and loop, not noise);
- the pilot's asymmetric-truncation hypothesis (200 epochs moved dense +1.27 and invariant -0.52 at $\ell = 100$);
- "training is not bit-reproducible" stated without scope (true of our loop only; the upstream loop is byte-identical per machine);
- the $\ell = 50$ inversion presented as a fresh, unexplained anomaly (it matches CLAIMS §4.10).

Each correction names the superseded value so that it cannot reappear.